

# Data Structures Notes: ADT Stack and its Operations

---

## Unit: ADT Stack

### 1. Abstract Data Type (ADT)

An **Abstract Data Type (ADT)** is a logical description of a data structure that specifies:

- What data is stored
- What operations can be performed
- It does **not** specify how the operations are implemented.

Example:

A **Stack ADT** defines operations such as:

- Push
- Pop
- Peek (Top)
- isEmpty
- isFull

without specifying whether the stack is implemented using arrays or linked lists.

---

## 2. Stack

### Definition

A **Stack** is a **Linear Data Structure** that follows the **LIFO (Last In First Out)** principle.

The last element inserted is the first element removed.

Example

```
Push 10
Push 20
Push 30
```

```
Stack
```

```
Top
↓
30
20
10
```

Pop →

30 removed first

---

## Real-Life Examples

- Stack of plates
  - Browser Back button
  - Undo operation in MS Word
  - Function Call Stack
  - Parentheses Matching
- 

## Stack Representation

### 1. Array Representation

```
#define MAX 100

int stack[MAX];
int top=-1;
```

#### Advantages

- Easy implementation
- Fast

#### Disadvantages

- Fixed size
- 

### 2. Linked List Representation

Each node stores

```
Data
Pointer to next node
```

#### Advantages

- Dynamic memory
- No overflow until memory is full

#### Disadvantages

- Extra memory for pointer
- 

## Basic Stack Operations

### 1. Push

#### Definition

Insert an element at the top of the stack.

#### Algorithm

```
PUSH(Stack, item)

1. If TOP == MAX-1
    Print "Overflow"
2. Else
    TOP = TOP + 1
    STACK[TOP] = item
3. Stop
```

---

### C Program

```
C

#include<stdio.h>

#define MAX 5

int stack[MAX];
int top=-1;

void push(int item)
{
    if(top==MAX-1)
    {
        printf("Stack Overflow\n");
        return;
    }

    top++;
    stack[top]=item;
}
```

## Example

Initially

```
Top=-1
```

Push(10)

```
10
↑Top
```

Push(20)

```
20
10
↑Top
```

Push(30)

```
30
20
10
↑Top
```

---

## Complexity

| Operation | Complexity |
|-----------|------------|
| Push      | O(1)       |

---

## 2. Pop

### Definition

Removes the top element.

---

### Algorithm

```
POP(Stack)
```

```
1. If TOP== -1
    Underflow
2. Else
    item = STACK[TOP]
    TOP = TOP-1
3. Return item
```

---

## C Code

```
C

int pop()
{
    if(top== -1)
    {
        printf("Stack Underflow\n");
        return -1;
    }

    return stack[top--];
}
```

---

## Example

### Stack

```
30
20
10
↑Top
```

### After Pop

```
20
10
↑Top
```

### Removed

```
30
```

---

## Complexity

$O(1)$

---

### 3. Peek (Top)

Returns the top element without deleting it.

---

#### Algorithm

PEEK(Stack)

1. If  $TOP == -1$   
Underflow
  2. Else  
Return  $STACK[TOP]$
- 

#### C Code

```
C

int peek()
{
    if(top== -1)
    {
        printf("Empty Stack\n");
        return -1;
    }

    return stack[top];
}
```

---

#### Complexity

$O(1)$

---

### 4. isEmpty()

## Algorithm

```
If TOP== -1
    TRUE
Else
    FALSE
```

## C Code

```
C

int isEmpty()
{
    return top== -1;
}
```

## Complexity

$O(1)$

---

## 5. isFull()

### Algorithm

```
If TOP==MAX-1
    TRUE
Else
    FALSE
```

## C Code

```
C

int isFull()
{
    return top==MAX-1;
}
```

## Complexity

$O(1)$

---

# Complete Stack Program

C

```
#include<stdio.h>

#define MAX 5

int stack[MAX];
int top=-1;

void push(int item)
{
    if(top==MAX-1)
    {
        printf("Overflow\n");
        return;
    }

    stack[++top]=item;
}

int pop()
{
    if(top==--1)
    {
        printf("Underflow\n");
        return -1;
    }

    return stack[top--];
}

int peek()
{
    if(top==--1)
        return -1;

    return stack[top];
}

void display()
{
    int i;

    if(top==--1)
    {
        printf("Empty Stack\n");
        return;
    }
}
```



```
        for(i=top;i>=0;i--)
            printf("%d\n",stack[i]);
    }

    int main()
    {
        push(10);
        push(20);
        push(30);

        display();

        printf("Popped=%d\n",pop());

        printf("Top=%d\n",peek());

        display();

        return 0;
    }
```

---

## Time Complexity of Stack Operations

| Operation | Complexity |
|-----------|------------|
| Push      | O(1)       |
| Pop       | O(1)       |
| Peek      | O(1)       |
| isEmpty   | O(1)       |
| isFull    | O(1)       |

---

## Applications of Stack

Stacks are widely used in:

- Function calls and recursion
- Undo/Redo operation
- Browser Back/Forward
- Parentheses Matching
- Expression Conversion

- Expression Evaluation
  - Syntax Parsing
  - Depth First Search (DFS)
- 

## Expression Conversion

Expressions can be written in three forms.

### Infix

Operator is between operands.

Example

A+B  
A+B\*C

---

### Prefix

Operator comes before operands.

Example

+AB  
\*+ABC

---

### Postfix

Operator comes after operands.

Example

AB+  
ABC\*+

---

## Operator Precedence

| Operator    | Priority |
|-------------|----------|
| $\wedge$    | 3        |
| $\ast / \%$ | 2        |
| $+ -$       | 1        |

## Infix to Postfix Conversion

### Algorithm

```
Scan expression left to right

If operand
    Output it

If '('
    Push

If ')'
    Pop until '('

If operator

    While stack not empty and
        precedence(top) >= precedence(current)

        Pop to output

    Push current operator

After scanning

Pop remaining operators
```

### Example

Convert

A+B\*C

| Input | Stack | Output |
|-------|-------|--------|
| A     |       | A      |
| +     | +     | A      |
| B     | +     | AB     |
| *     | +*    | AB     |
| C     | +*    | ABC    |
| End   |       | ABC*+  |

## Result

ABC\*+

## Example

### Convert

(A+B) \*C

### Output

AB+C\*

## Complexity

O (n)

## C Program (Infix to Postfix)

C

```
#include<stdio.h>
#include<ctype.h>
```

```

char stack[100];
int top=-1;

void push(char c)
{
    stack[++top]=c;
}

char pop()
{
    return stack[top--];
}

int precedence(char c)
{
    if(c=='+'||c=='-')
        return 1;
    if(c=='*'||c=='/')
        return 2;
    if(c=='^')
        return 3;

    return 0;
}

int main()
{
    char exp[]="A+B*C";
    char post[100];
    int i=0,j=0;

    while(exp[i])
    {
        char ch=exp[i];

        if(isalnum(ch))
            post[j++]=ch;

        else
        {
            while(top!=-1 &&
                precedence(stack[top])>=precedence(ch))
                post[j++]=pop();

            push(ch);
        }

        i++;
    }

    while(top!=-1)

```

```
post[j++] = pop();

post[j] = '\0';

printf("%s", post);
```

---

## Postfix Evaluation

### Example

23\*54\*+

### Equivalent

$(2 \times 3) + (5 \times 4)$

---

## Algorithm

Scan expression

If operand

Push

If operator

op2=Pop()

op1=Pop()

result=op1 operator op2

Push(result)

Final answer = Pop()

---

## Example

### Expression

23\*54\*+

| Symbol | Stack |
|--------|-------|
| 2      | 2     |
| 3      | 2 3   |
| *      | 6     |
| 5      | 6 5   |
| 4      | 6 5 4 |
| *      | 6 20  |
| +      | 26    |

Answer

26

Complexity

$O(n)$

## C Program (Postfix Evaluation)

C

```
#include<stdio.h>
#include<ctype.h>

int stack[100];
int top=-1;

void push(int x)
{
    stack[++top]=x;
}

int pop()
{
    return stack[top--];
}
```

```

}

int main()
{
    char exp[]="23*54*+";
    int i=0;

    while(exp[i])
    {
        if(isdigit(exp[i]))
        {
            push(exp[i]-'0');
        }
        else
        {
            int b=pop();
            int a=pop();

            switch(exp[i])
            {
                case '+': push(a+b); break;
                case '-': push(a-b); break;
                case '*': push(a*b); break;
                case '/': push(a/b); break;
            }
        }

        i++;
    }

    printf("Answer = %d",pop());

    return 0;
}

```

## Complexity Analysis

| Algorithm | Best | Average | Worst |
|-----------|------|---------|-------|
| Push      | O(1) | O(1)    | O(1)  |
| Pop       | O(1) | O(1)    | O(1)  |
| Peek      | O(1) | O(1)    | O(1)  |
| isEmpty   | O(1) | O(1)    | O(1)  |
| isFull    | O(1) | O(1)    | O(1)  |



| Algorithm          | Best   | Average | Worst  |
|--------------------|--------|---------|--------|
| Infix → Postfix    | $O(n)$ | $O(n)$  | $O(n)$ |
| Postfix Evaluation | $O(n)$ | $O(n)$  | $O(n)$ |

## Examination Viva Questions

1. What is an Abstract Data Type (ADT)?
2. What is a stack?
3. Why is a stack called a LIFO data structure?
4. Differentiate between stack overflow and stack underflow.
5. Explain the Push and Pop operations with examples.
6. List any five real-life applications of stacks.
7. Convert the infix expression  $(A+B) * (C-D)$  into postfix.
8. Evaluate the postfix expression  $82/53*+.$
9. Why are stacks used in recursion?
10. What is the time complexity of Push, Pop, and Peek operations?
11. Differentiate between infix, prefix, and postfix expressions.
12. Explain the algorithm for infix-to-postfix conversion.
13. Describe the algorithm for postfix expression evaluation.
14. How is operator precedence handled during expression conversion?
15. Compare array-based and linked-list implementations of a stack.